■ Project 1: "TerraSpectra" — Hyperspectral Crop Disease Forecasting

Master System Specification & Action Blueprint

Project Overview & System Description

■ Project Title

Project 1 — "TerraSpectra": Hyperspectral Crop Disease Forecasting

■ Domain & Specialization

Precision Agriculture & Advanced Computer Vision

■■ Problem Statement

Traditional satellite crop monitoring relies on standard 3-channel RGB (Red, Green, Blue) or simple 4-channel Multispectral imagery (such as NDVI using Near-Infrared). These conventional models operate on a fatal flaw: **they can only detect crop diseases after leaves turn yellow or brown (foliar chlorosis and necrosis)**. By the time visual symptoms appear to human eyes or standard 2D Convolutional Neural Networks (2D-CNNs), internal cellular structures of the plants have already collapsed, making harvest yield loss unrecoverable.

Furthermore, standard 2D-CNN architectures treat image input as flat 2D spatial matrices with a few color channels. They are fundamentally incapable of processing the massive 3D spatial-spectral depth of Hyperspectral Data Cubes, which contain hundreds of narrow, contiguous spectral bands.

■ Real-World Use Case & Vision

An agricultural analyst monitors a 1,000-acre commercial farm via the **TerraSpectra** platform. The system ingests **Hyperspectral Satellite Data Cubes** capturing **200+ contiguous spectral bands of light** across the electromagnetic spectrum (from 400 nm to 2500 nm), far beyond human visual perception.

The backend **Hybrid 3D-CNN + Vision Transformer (ViT)** AI model analyzes subtle chemical changes in the plants' chlorophyll-a/b absorption, anthocyanin accumulation, and cellular water retention. The TerraSpectra GIS dashboard automatically highlights a specific 5-acre zone in glowing red, predicting a **fungal blight outbreak three weeks before any visible symptoms appear on the leaves**. This early warning allows farm managers to execute hyper-targeted, localized preventative treatment, saving 95% of pesticide costs and preserving crop yield.

■■ Core System Modules

- **Geospatial Data Pipeline (Python, Rasterio, GDAL, HDF5, NetCDF4):** Ingests, calibrates, and normalizes multi-gigabyte hyperspectral data cubes (from NASA Hyperion, ESA Sentinel-6, or AVIRIS airborne sensors).
- **3D-CNN & ViT Hybrid AI Model (PyTorch):** A complex neural network utilizing 3D convolutions to capture spatial features alongside a Vision Transformer (ViT) self-attention engine to process deep inter-spectral band correlations.
- **High-Throughput Inference API (FastAPI):** REST API capable of GPU-accelerated image chunking, sliding-window tiling, and real-time prediction raster generation.

- **Interactive GIS Dashboard (React 18, Deck.gl, Mapbox GL JS):** A professional 3D map interface rendering predicted disease heatmaps overlaid on satellite topography with interactive spectral analytics.

Week-1

Track	Core Requirement	Technologies
ML Engineering & Data Pipeline	**Data Preparation:** Use Rasterio to parse hyperspectral data cubes (HDF5 / GeoTIFF formats). Perform dimensionality reduction (PCA) on 200+ spectral bands down to 32 components.	Python, Rasterio, h5py, Scikit-learn, NumPy
Frontend & GIS Visualization	**Map Scaffolding:** Build React app. Integrate Deck.gl and Mapbox to render the base geographical farm layers.	React 18, TypeScript, Vite, Deck.gl, Mapbox GL JS, TailwindCSS

Phase Timeline

Phase	Phase Name	Focus Area & Description
P1	**Phase 1: Research Topics Needed**	Spectral physics, plant pathology, HDF5/GeoTIFF specifications, WebGL GIS, CUDA hardware.
P2	**Phase 2: Data Assets Needed**	Identify data sources, acquire satellite datasets, obtain web GIS API tokens, collect DEM elevation tiles.
P3	**Phase 3: Software Frameworks to Research**	Geospatial ML Python backend libraries, React WebGL GIS frontend dependencies, CI/CD tools.
P4	**Phase 4: Advanced Research & Algorithm Headings**	PCA derivations, Vegetation Indices (PRI/WBI/SIPI), 3D Convolutions, Transformer self-attention, MLOps drift.
P5	**Phase 5: Detailed Execution & Validation Checklist**	Data ingestion, PCA pipeline, REST API tiling, React scaffolding, WebGL layers, E2E testing.

Phase 1: Theoretical Research Topics

Research Category	Headings & Concepts to Research
Backend Data Science	• Hyperspectral Wavelength Ranges (VIS, Red Edge, NIR, SWIR) • Plant Pathology Biomarkers & Cellular Collapse • Red Edge Spectral Blue Shift Mechanics • Radiometric & Atmospheric Calibration Models • Geospatial File Standards (HDF5, GeoTIFF, NetCDF) • Coordinate Reference Systems (EPSG:4326 vs EPSG:3857) • Spectral Preprocessing & Continuum Removal • CUDA GPU Memory Allocation & Tensor Tiling Mechanics • Security & Farm Spatial Privacy Protocols
Frontend GIS	• WebGL GIS Map Rendering Engines • Deck.gl Layer Architecture & Compositing • Viewport Camera Synchronization Mechanics • Interactive Canvas Pixel Picking & Spectral Charts • WebGL Context & Performance Optimization • Client-Side Caching & Vector Tile Memory Management

Phase 2: Data Assets Needed

Component Layer	Assets & Tokens to Search & Acquire
Backend Data Science	• Airborne Hyperspectral Datacubes (200+ Spectral Bands) • Spaceborne Satellite Datacubes • Multispectral Satellite Calibration Imagery • Synthetic Datacube Generator Scripts • Spectroradiometer Ground Truth Leaf Measurement Datasets
Frontend GIS	• Web GIS Tile Provider Access Tokens • Satellite Basemap Style URIs • Farm Boundary Vector Polygons (GeoJSON) • 3D Digital Elevation Models (DEM Terrain-RGB)

Phase 3: Packages & Modules to Research and Install

Environment	Framework & Module Research Headings

Backend Data Science (Python)	• Geospatial Raster & Datacube I/O Packages • Machine Learning & Dimensionality Reduction Libraries • Deep Learning Frameworks & 3D Convolutions • Vision Transformer Architecture Backbones • Explainable AI (XAI) & Feature Attribution Tools • High-Throughput Async REST API Frameworks • Vector GIS Data Engines & Signal Processing Libraries • MLOps & Model Performance Tracking Tools • Automated Testing & Async Code Verification Frameworks
Frontend GIS (Node.js)	• React UI Component Framework & State Management • WebGL GIS Engine & High-Performance Map Layers • Satellite Basemap Provider SDKs & Viewport Controls • Interactive Spectral Curve Charting Libraries • UI Icon Sets, Utility Styling & Build Bundlers • End-to-End (E2E) & Component Testing Frameworks

Phase 4: Advanced Research & Algorithm Headings

Research Domain	Core Headings & Sub-Headings to Study
PCA Band Reduction	• Spatial Tensor Reshaping (3D to 2D) • Mean Centering & Covariance Matrix Construction • Eigenvalue Decomposition & Spectral Sorting • Cumulative Explained Variance Evaluation ($\ge 98.5\%$)
Vegetation Indices	• Photochemical Reflectance Index (PRI) • Water Band Index (WBI) • Structure Insensitive Pigment Index (SIPI)
3D Deep Learning & Attention	• 3D Spatial-Spectral Convolutional Kernels • Vision Transformer Multi-Head Self-Attention
Radiometric Calibration	• Atmospheric Radiative Transfer Models (6S & FLAASH) • Atmospheric Water Vapor Noise Suppression
Continuum Removal & Normalization	• Convex Hull Normalization & Band Depth Isolation
Explainable AI (XAI) & MLOps	• Integrated Gradients Spectral Attribution Scores • Seasonal Illumination & Concept Drift Monitoring

Phase 5: Detailed Execution & Validation Checklist

■■ Backend Data Science & Pipeline Validation Checklist

1. Data Ingestion & Environment Verification

- [ ] Confirm Python virtual environment activation and package dependency resolution.
- [ ] Verify installation of raster, array slicing, machine learning, and API server packages.
- [ ] Ingest sample 3D Hyperspectral Data Cubes (HDF5 / GeoTIFF / NetCDF formats).
- [ ] Validate array tensor spatial-spectral dimensions `(Height, Width, Bands)`.
- [ ] Verify GeoTIFF GeoKey metadata and Coordinate Reference Systems (EPSG:4326 / EPSG:3857).

2. Dimensionality Reduction & Spectral Preprocessing

- [ ] Implement spatial tensor reshaping from 3D volume `(H, W, B)` to 2D matrix `(H*W, B)`.
- [ ] Execute mean-centering normalization and sample covariance matrix construction.
- [ ] Perform eigenvalue decomposition and spectral principal component sorting.
- [ ] Compress spectral depth from 200+ bands down to top 32 principal components.
- [ ] Confirm cumulative retained variance ratio meets or exceeds 98.5%.
- [ ] Compute narrow-band vegetation index maps (PRI, WBI, SIPI).

3. Base API Scaffolding & Tiling Endpoint Verification

- [ ] Scaffold async REST API server with CORS middleware enabled.
- [ ] Implement `/health` status endpoint returning 200 OK health payload.
- [ ] Implement `/api/v1/pca/reduce` async route for real-time raster array chunk processing.
- [ ] Validate request/response JSON schema DTO models using Pydantic.
- [ ] Test API throughput and Swagger OpenAPI documentation (`http://localhost:8000/docs`).

4. Security, MLOps & Performance Benchmarking

- [ ] Benchmark CUDA GPU memory usage during sliding-window array chunk processing.
- [ ] Verify API security headers, CORS origins, and token authorization.
- [ ] Execute automated unit test suite (`pytest`) covering array preprocessing functions.

■ Frontend WebGL GIS Validation Checklist

1. Map Scaffolding & Component Setup

- [ ] Initialize React 18 application shell with Vite bundler and TypeScript configuration.
- [ ] Configure TailwindCSS utility styling and dark-mode layout container.
- [ ] Select Web GIS satellite tile provider and acquire public access token.
- [ ] Initialize base interactive 3D satellite map container.
- [ ] Verify camera viewport viewstate controls (latitude, longitude, zoom, pitch, bearing).

2. Base Layer Composition & Data Overlays

- [ ] Instantiate Deck.gl WebGL layer canvas overlaid on satellite basemap.

- [ ] Configure Deck.gl `GeoJsonLayer` for farm field boundary vector rendering.
- [ ] Configure Deck.gl `BitmapLayer` for prediction heatmap raster overlay rendering.
- [ ] Test pyramid tile loading via Deck.gl `TileLayer` for zooming performance.
- [ ] Import Mapbox Terrain-RGB DEM elevation tiles for 3D topography rendering.

3. Interactive Analytics & Spectral Visualization

- [ ] Implement canvas hover and click pixel picking event handlers.
- [ ] Extract clicked pixel lat/long coordinates and 224-band spectral reflectance values.
- [ ] Render continuous 224-band spectral reflectance line chart using Plotly.js canvas.
- [ ] Verify WebGL canvas context loss handling and repaint loop optimization.

4. Frontend Quality & Performance Benchmarking

- [ ] Verify responsive layout across desktop, tablet, and mobile displays.
- [ ] Execute component unit tests (`vitest`) and verify clean build compilation.